

Binary Search Tree

A Binary Search Tree (BST) is a hierarchical data structure that maintains sorted data to enable efficient searching, insertion, and deletion operations. Mastering BST invariants and time complexities is crucial for data structures and algorithms examinations.

Definition and Invariants

- A binary search tree is either empty or consists of a node and two disjoint subtrees called left and right subtrees.
- The BST property states that the key in any node must be greater than all keys in its left subtree.
- The key in any node must be less than or equal to all keys in its right subtree.
- Both the left and right subtrees must also be valid binary search trees.

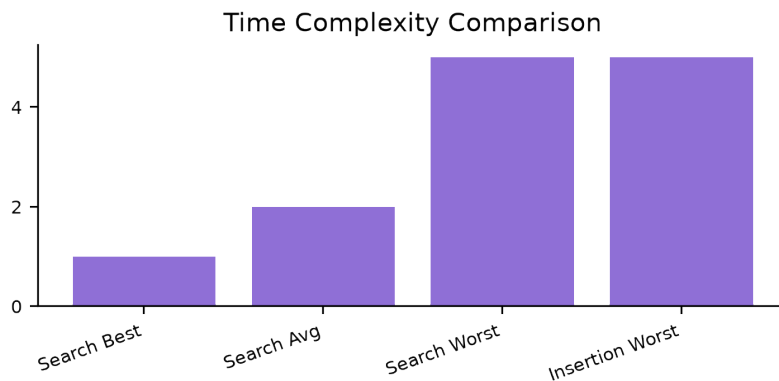
Core Operations and Complexity

- Search, insertion, and deletion depend on the height of the tree.
- In a balanced BST, operations run in logarithmic time due to divide-and-conquer traversal.
- In the worst-case scenario where the tree becomes degenerate, operations degrade to linear time.
- Space complexity is proportional to the number of nodes stored in the tree.

$$T_{avg}(n) = \mathcal{O}(\log n)$$

$$T_{worst}(n) = \mathcal{O}(n)$$

$$S(n) = \mathcal{O}(n)$$



Deletion Algorithm Process

- Case 1: Node to delete is a leaf node; simply remove it by updating its parent's pointer to null.
- Case 2: Node has only one child; bypass the node by linking its parent directly to its child.
- Case 3: Node has two children; find its in-order successor or predecessor, copy its value, and delete that successor node.

BST Deletion Cases



Tree Traversals

- In-order traversal visits the left subtree, the root, and then the right subtree, producing sorted output.
- Pre-order traversal visits the root first, followed by the left subtree and right subtree, useful for cloning trees.
- Post-order traversal visits left and right subtrees before the root, commonly used for deletion and bottom-up metric calculations.

Applications

- Used to implement dynamic sets and associative arrays.
- Serves as the underlying data structure for sets and maps in many standard library implementations.
- Acts as the foundational concept for self-balancing variants like AVL and Red-Black trees.

Key Terms

BST Invariant

The structural rule ensuring left subtree keys are smaller and right subtree keys are larger than the parent node.

In-order Successor	The node with the smallest key that is greater than the key of a given node.
Degenerate Tree	A pathological BST where each parent has only one child, resembling a linked list.
Balanced Tree	A tree where the heights of the left and right subtrees of any node differ by at most one.
In-order Traversal	A traversal strategy that visits nodes in ascending order of their keys.

Exam Tips

- Always state both average and worst-case time complexities when asked about BST operations, and explain why the worst-case happens.
- Be prepared to trace step-by-step node deletion, especially Case 3 involving the in-order successor.
- Remember that an in-order traversal of a valid BST always yields a sorted sequence of elements.